

Unit II- Software Requirements Engineering

Software Requirements Engineering (SRE) is the systematic process of identifying, documenting, analyzing, validating, and managing the requirements of a software system.

A software project begins with a simple question:

“What does the customer or user need from the software?”

Requirements Engineering helps convert this need into a clear specification that developers, testers, customers, and other stakeholders can understand.

Why Requirements Engineering is Important

Requirements Engineering is one of the most important activities in software development.

Importance

1. Understands customer needs

It helps developers understand exactly what customers expect.

2. Reduces development errors

Clear requirements reduce misunderstanding between customers and developers.

3. Controls project scope

Requirements define what is included and what is not included in the project.

4. Helps in project estimation

Requirements help estimate:

Cost

Time

Resources

Number of developers

5. Helps testing

Test cases are developed based on requirements.

6. Reduces project failure

Many software projects fail because requirements are incomplete, unclear, or continuously changing.

7. Improves communication

The SRS provides a common document for:

Customer

Developer

Tester

Project manager

Business analyst

1. What is a Software Requirement?

A **software requirement** is a description of a service, function, capability, or constraint that a software system must satisfy.

Requirements can describe:

What the system should do

How the system should behave

Restrictions or limitations

Quality characteristics of the system

Good requirements are clear, complete, consistent, feasible, testable and traceable.

Example:

Suppose a college wants to develop an Online Student Attendance System.

The college may require:

- Teachers should be able to mark attendance.
- Students should be able to view their attendance.
- The system should generate attendance reports.
- Only authorized users should access student records.
- The system should respond quickly.

These statements are software requirements

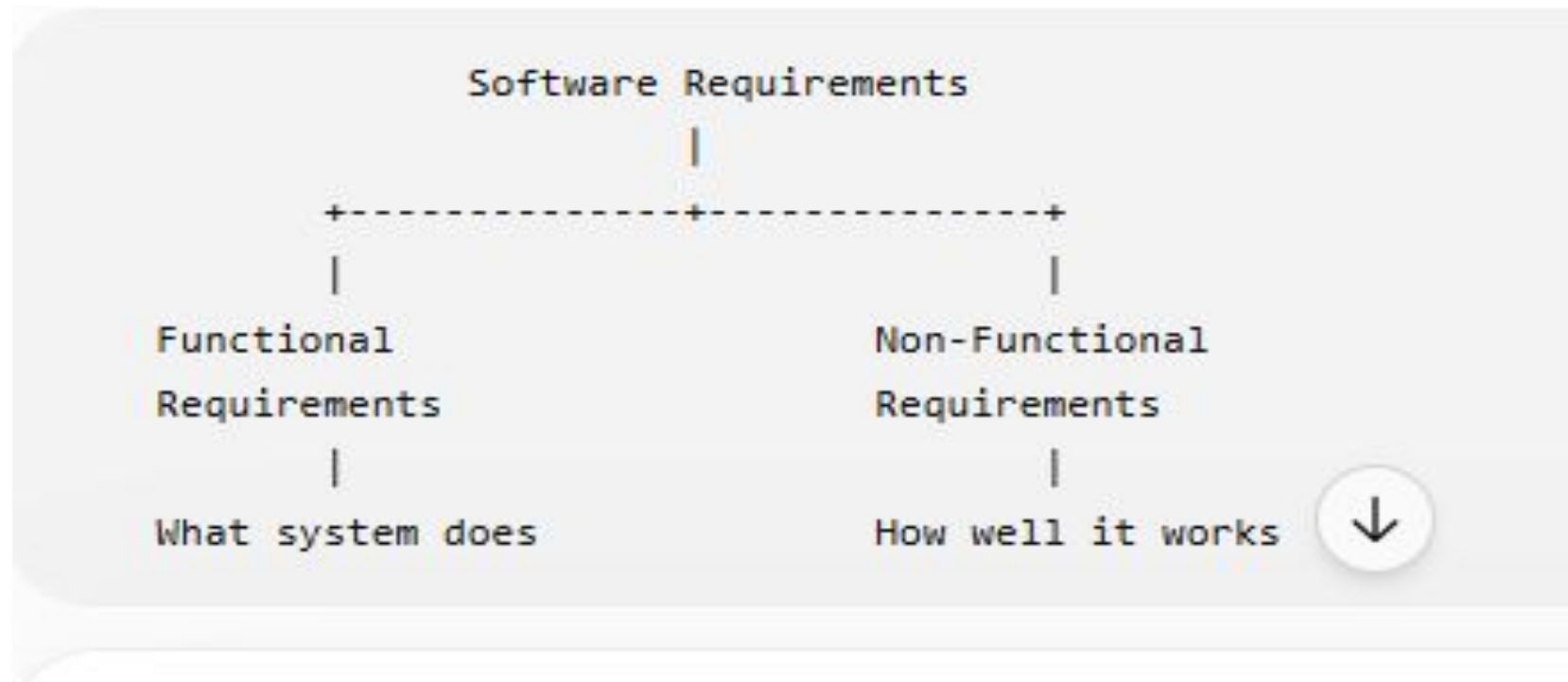
Types of Software Requirements

Software requirements are broadly divided into:

Software requirements are broadly divided into:

Types of Software Requirements

Software requirements are broadly divided into:



Functional vs Non-functional Requirements

Functional requirements: describe specific services, inputs, outputs and system behaviour.

Non-functional requirements: describe quality attributes, constraints and performance expectations.

Functional = WHAT the system does.

Non-functional = HOW WELL the system performs.

Functional Requirements

Definition

Functional requirements describe the specific functions or services that a software system must provide.

They describe **what the system should do**.

They specify:

- Inputs
- Outputs
- Calculations
- Processing
- User interactions
- System behaviour

Characteristics of Functional Requirements

A good functional requirement should be:

Clear

Complete

Correct

Consistent

Testable

Unambiguous

Functional Requirements — Real Example: Online Banking System

Functional requirements may include:

User shall be able to create an account.

User shall be able to log in.

User shall be able to check account balance.

User shall be able to transfer money.

User shall be able to download a bank statement.

System shall generate transaction confirmation.

Functional Requirements — Real Example

Online Food Delivery System:

- Customer can register/login.
- Customer can search restaurants and menus.
- Customer can add/remove items from cart.
- System can calculate bill, tax and delivery charge.
- Customer can place an order and make payment.
- Restaurant can accept/reject an order.
- Customer can track order status.

Examples of Functional Requirements Online Shopping System

Function	Requirement
Registration	User shall be able to register
Login	User shall be able to log in
Search	User shall be able to search products
Cart	User shall be able to add products
Payment	User shall be able to make payment
Order	System shall generate an order
Tracking	User shall be able to track delivery

Non-functional Requirements

Definition

Non-functional requirements describe the quality attributes, constraints, and performance characteristics of a software system.

They answer:

“How well should the system perform its functions?”

Examples include:

- Performance
- Security
- Reliability
- Availability
- Usability
- Scalability
- Maintainability
- Portability

Types of Non-functional Requirements

Performance

Defines how quickly the system should respond.

Example

The system shall display search results within 2 seconds.

Security

Defines requirements for protecting data and preventing unauthorized access.

Example

Passwords must be securely stored.

Users must authenticate before accessing personal data.

Only authorized staff can modify student marks.

Reliability

Defines how consistently the system operates without failure.

Example

The system should successfully process 99.5% of valid transactions.

Availability

Defines how much time the system should be operational.

Example

The online banking system should be available 99.9% of the time.

Usability

Describes how easy the system is to learn and use.

Example

A new student should be able to register without formal training.

Scalability

Describes the ability of the system to handle increasing workload.

Example

A college application may initially have 5,000 students but should be capable of supporting 50,000 students in the future.

Maintainability

Describes how easily software can be modified, corrected, or enhanced.

Example

The system should use modular components so that individual modules can be modified without affecting unrelated modules

Portability

Describes the ability of software to operate on different platforms.

Example

An application should work on:

Windows

Linux

Android

iOS

Example

Consider a banking application.

Functional requirement:

User shall be able to transfer money.

Non-functional requirements:

The transaction should be completed within 3 seconds.

The system should be available 99.9% of the time.

Customer data must be encrypted.

Thus:

Functional = What the system does

Non-functional = How well or under what constraints it does it

Non-functional Requirements — Real Example

Performance: 95% of normal requests should respond within 2 seconds.

- Availability: service should be available 99.9% of the month.
- Security: passwords must be stored securely; payment data must be protected.
- Usability: a new customer should be able to place an order without training.
- Scalability: system should support increased users during peak meal times.
- Maintainability: code and documentation should support easy updates.

Functional vs Non-functional Requirements

Functional Requirements

Describe what the system does

Concerned with system services

Usually feature-oriented

Can describe inputs and outputs

Example: Login

Example: Generate report

Non-functional Requirements

Describe how well the system works

Concerned with quality attributes

Usually quality/constraint-oriented

Can describe performance and security

Example: Login must complete within 2 seconds

Example: Report must be generated within 5 seconds

Easy way to remember

Functional = WHAT?

Non-functional = HOW WELL?

2. Software Requirements Document (SRS)

SRS = Software Requirements Specification/Document.

A Software Requirements Document (SRD)—often called a Software Requirements Specification (SRS)

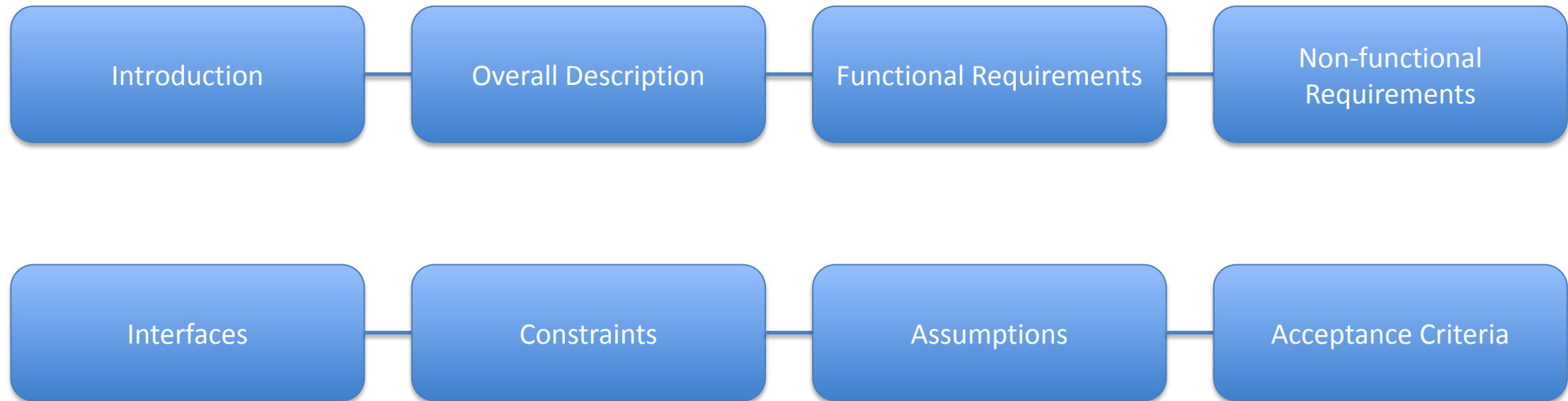
It is an agreed document describing system services, constraints and quality requirements.

SRS document is a complete blueprint that describes a software product's behavior, features, and constraints before development begins. It acts as an official agreement between the client and the development team

Purpose: common understanding among customer, users, analysts, developers and testers.

Typical contents: introduction, overall description, functional requirements, non-functional requirements, interfaces, constraints, assumptions and acceptance criteria.

Typical SRS Structure



Requirements Specification

Requirements specification converts user needs into precise statements that can be reviewed and tested.

Example (functional): “The system shall send an OTP to the registered mobile number during login.”

Example (non-functional): “The OTP service shall deliver the OTP within 10 seconds in 95% of attempts.”

Use measurable language instead of vague words such as fast, easy, user-friendly or sufficient.

Components of an SRS Document

- **Introduction and Purpose:** Explains why the software is being built, the project scope, and the target audience.
- **Functional Requirements:** Lists specific system features, user actions, inputs, and expected outputs.
- **Non-Functional Requirements (NFRs):** Defines quality attributes like system performance, security, reliability, and scalability.
- **External Interfaces:** Describes how the software connects with users, hardware, databases, and external APIs.
- **Constraints and Assumptions:** Lists design limits, budget boundaries, and technical dependencies.

SRS Format



Introduction

- Purpose
- Document Conventions
- Product Scope
- Reference

Overall Description

- Product Perspective
- Product Functions
- User Classes and Characteristics
- Operating Environment
- Design and Implementation Constraints
- User Documentation
- Assumptions and Dependencies

External Interface Requirements

- User Interfaces
- Hardware Interfaces
- Software Interfaces
- Communications Interfaces

System Features

- System Feature 1
- System Feature 2
(and so on)

Other Nonfunctional Requirements

- Performance Requirements
- Safety Requirements
- Security Requirements
- Software Quality Attributes
- Business Rules

Other Requirements

- Appendix A: Glossary
- Appendix B: Analysis Models

Types of reader for requirement specification



System Customers

Specify the requirements and read them to check that they meet their needs. Customer specify changes to the requirements .

Managers

Use the Requirements Document to plane a bid for the system and to plan system development process.

System Engineers

Use the requirements to understand what system is to be deployed.

System test engineers

Use the requirements to develop validation tests for the system

System Maintains engineers

Use the requirements to understand the system and the relationship between its parts

what are the benefits of a great SRS?

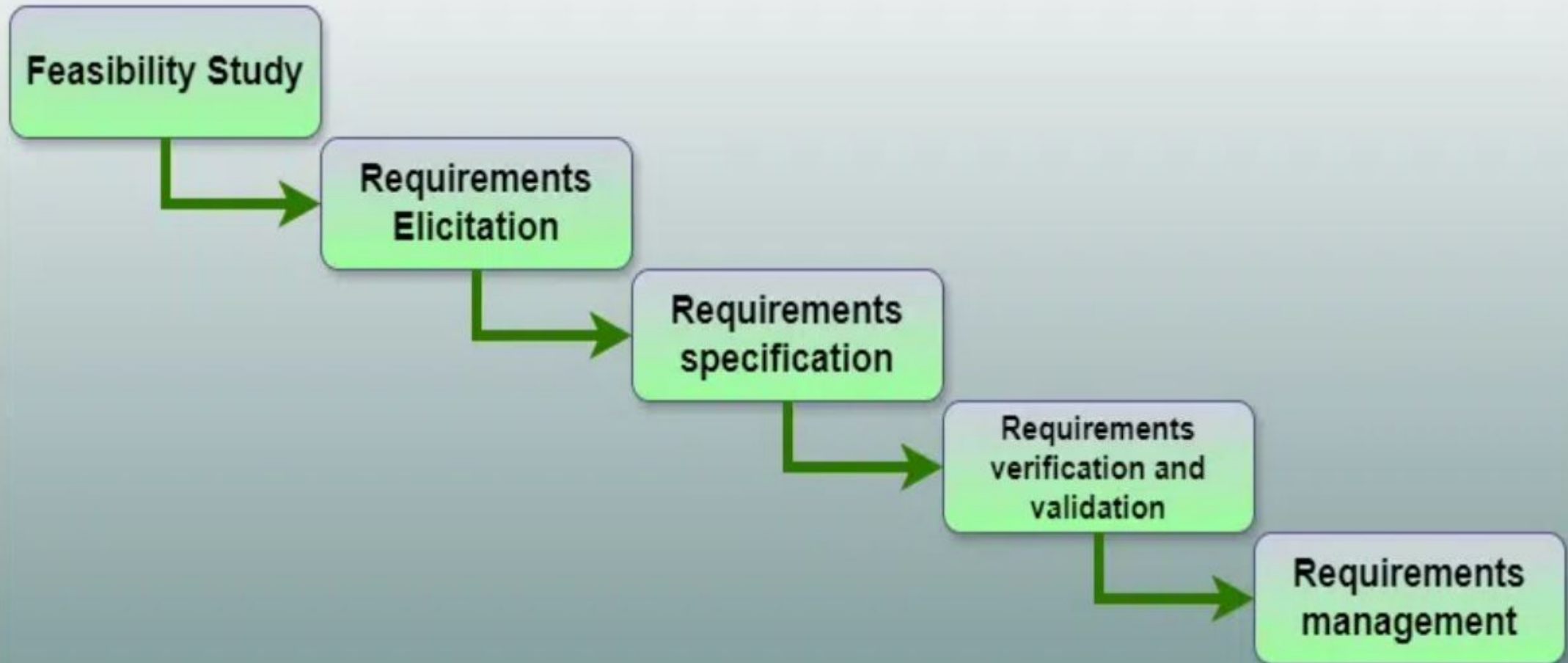
The IEEE 830 standard defines the benefits of a good SRS:

- Establish the basis for agreement between the customers and the suppliers on what the software product is to do.
- Reduce the development effort.
- Provide a basis for estimating costs and schedules.
- Provide a baseline for validation and verification.
- Serve as a basis for enhancement.

Requirements Engineering Process

Requirements Engineering is the process of identifying, eliciting, analyzing, specifying, validating, and managing the needs and expectations of stakeholders for a software system.

Requirements Engineering Process



1. Feasibility Study

The feasibility study mainly concentrates on below five mentioned areas below. Among these Economic Feasibility Study is the most important part of the feasibility analysis and the Legal Feasibility Study is less considered feasibility analysis.

- **Technical Feasibility:** Assesses whether the organization has the right hardware, software, and technical skills to build the system.
- **Economic Feasibility:** Weighs project development and maintenance costs against financial and operational benefits.
- **Operational Feasibility:** Examines how well the new software will solve business problems and fit user workflows and company culture.
- **Legal and Schedule Feasibility:** Checks compliance with regulations, data privacy laws, and determines if project timelines are realistic

2. Requirements Elicitation

Requirements elicitation is the foundational process of gathering, discovering, and defining user needs, expectations, and constraints for a software system

Activities in Elicitation

- **Stakeholder Identification:** Find everyone who affects or is affected by the software, including end-users, managers, and clients.
- **Information Gathering:** Collect details about current business processes, workflows, and problems.
- **Classification and Negotiation:** Organize raw data, resolve conflicting user requests, and prioritize features.
- **Documentation:** Record findings in formats like a Software Requirements Specification (SRS).



Main Requirement Elicitation Techniques used

Common Elicitation Techniques

- **Interviews:** Direct, one-on-one or group sessions to ask structured or open-ended questions.
- **Workshops:** Collaborative group meetings that build consensus and hash out conflicting needs.
- **Observation:** Shadowing users in their real work environment to see unstated or hidden habits.
- **Prototyping:** Building early mock-ups or models so users can test ideas visually and give feedback.
- **Brainstorming:** Creative group sessions meant to spark innovative features and new solutions.
- **Document Analysis:** Reviewing existing reports, manuals, or policies to extract legacy rules

3. Software Requirement Specification: In this phase, the gathered and analyzed requirements are documented formally. The Software Requirement Specification (SRS) document is created, which serves as a blueprint for the development process. The SRS includes the following components:

- 1. Functional Requirements:** Descriptions of what the system should do in terms of specific functions and features.
- 2. Non-functional Requirements:** Constraints and qualities the system must possess (e.g., performance, security).
- 3. User Interfaces:** Descriptions of how users will interact with the system.
- 4. Data Requirements:** Details about the data the system will store, process, and manage.

4. Requirements Verification and Validation

Verification:. It refers to the set of tasks that ensures that the software correctly implements a specific function. Verification is checking that the requirements are complete, consistent, and accurate. It involves reviewing the requirements to ensure that they are clear, testable, and free of errors and inconsistencies. This can include reviewing the requirements document, models, and diagrams

Software Requirement Validation: This step involves validating the documented requirements to ensure they accurately represent the stakeholders' needs and expectations. The aim is to identify and rectify any errors or misunderstandings before development begins. Techniques used for requirement validation include:

- 1. Reviews:** Expert reviews and walkthroughs to identify issues in the SRS.
- 2. Prototyping:** Building a working model of the software to validate requirements.
- 3. Simulation:** Simulating the software's behaviour to test its alignment with requirements.

5.Requirements Management

Requirement management is the process of analyzing, documenting, tracking, prioritizing, and agreeing on the requirement and controlling the communication with relevant stakeholders.

Requirement management involves

- Tracking requirements
- Managing changes
- Maintaining versions
- Analyzing the impact of changes
- Maintaining traceability

Simple Flow

Feasibility Study → Elicitation → Specification → Validation → Requirement Management

4. Risk Management

A **risk** is an uncertain event or condition that, if it occurs, can have a positive or negative effect on a project.

Risk management is the process of identifying, analyzing, and controlling uncertain events that may negatively affect a software project.

In software projects, risks may affect:

- Project cost
- Project schedule
- Product quality
- Scope and requirements
- Resources
- Technology
- Customer satisfaction
- Security and performance

Types of Software Project Risks

1. Technical Risks

Risks related to technology, architecture, tools, or implementation.

Examples:

New technology may not work as expected.

Integration with existing systems may fail.

Performance requirements may not be achieved.

Software may have serious security vulnerabilities.

2. Project Risks

Risks affecting project planning and execution.

Examples:

Unrealistic deadlines

Insufficient resources

Poor estimation

Staff turnover

Delays in development

3. Business Risks

Risks associated with the business environment.

Examples:

Change in market requirements

Competitor releases a similar product

Customer changes priorities

Project becomes financially unviable

4. Requirements Risks

Risks caused by unclear, incomplete, or changing requirements.

Examples:

- Customer does not clearly define requirements.
- Requirements change frequently.
- Different stakeholders have conflicting expectations.

5. Organizational Risks

Risks caused by the organization.

Examples:

- Lack of management support
- Organizational restructuring
- Budget reductions
- Poor communication between departments

6. External Risks

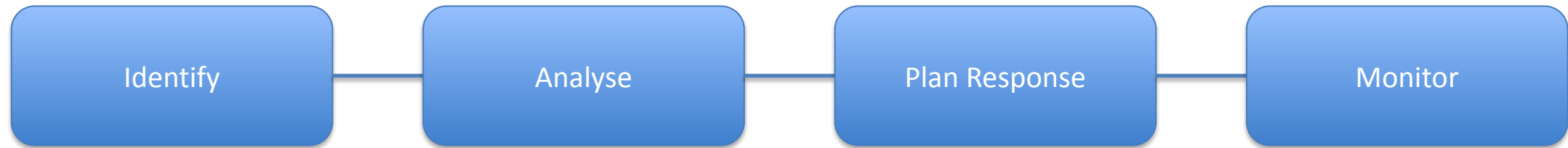
Risks outside the direct control of the development team.

Examples:

- Government regulations
- Economic changes
- Natural disasters
- Supplier problems
- Changes in external services

Risk Management Process

Risk management is a systematic process of identifying, analyzing, responding to, and monitoring risks.



1. Risk Identification

This first step is easy but powerful: just name the risk. In the process of risk management in software engineering, this to explore the possible failure points across the technology, people, and processes.

Teams often use:

Brainstorming sessions

Risk workshops

Historical project analysis

Stakeholder interviews

Technical reviews

A **risk register** is generally maintained to record identified risks.

Typical risk register fields:

Risk	Probability	Impact	Priority	Response
Developer leaves	Medium	High	High	Cross-training
Requirement changes	High	Medium	High	Change-control process
Technology failure	Low	High	Medium	Prototype early

2.Risk analysis

It is the process of evaluating and understanding the potential impact and likelihood of identified risks on an organization.

This helps determine how serious a risk is and how to best manage or mitigate it. Risk Analysis involves evaluating each risk's probability and potential consequences to prioritize and manage them effectively.

Risk analysis determines:

How likely the risk is to occur

How serious its consequences will be

Qualitative Analysis

Risks are categorized as:

High

Medium

Low

Quantitative Analysis

Teams assign numerical values to:

Probability of occurrence

Financial impact

Schedule impact

Risk Planning

After you have identified the most important risks, you have to determine how to respond to the risks. Common responses to risk include:

- Avoiding the risk:** Change the plan to eliminate the risk.

Example: Avoid an unstable technology by selecting a proven technology.

- Mitigating/Reducing the risk:** Do something to reduce the impact or probability.

Example: Train multiple developers so the project does not depend on one person.

- Transferring the risk:** outsource or share the risk (e.g., using a cloud provider for infrastructure, etc.).

Example: Purchase insurance or outsource a specialized activity.

- Accepting the risk:** live with the risk if it is minor or unavoidable.

Example: Accepting a small possibility of a minor delay.

Risk Monitoring

It involves continuously tracking and overseeing identified risks to assess their status, changes

Risks must be continuously monitored because:

New risks can appear.

Existing risks can become more serious.

Probability can change.

Risk-response actions may fail.

Project Planning

Definition

Project planning is the process of defining the project's objectives, activities, resources, schedule, cost, risks, and responsibilities before and during development.

The purpose is to create a clear plan for achieving project objectives.

Objectives of Project Planning

Project planning aims to:

- Define project scope.
- Identify required activities.
- Estimate effort and cost.
- Allocate resources.
- Develop a project schedule.
- Identify risks.
- Assign responsibilities.
- Establish quality standards.
- Monitor project progress.
- Ensure that the project meets customer requirements.

Major Components of Project Planning

1. Scope planning

Defines what is included and excluded from the project.

2. Effort estimation

Determines how much human effort is required.

Usually measured in:

Person-months / Person-hours / Person-days

3. Cost estimation

Determines the expected project cost.

Costs may include:

- Employee salaries
- Hardware
- Software licenses
- Cloud infrastructure
- Training
- Testing
- Travel
- Office expenses

4. Resource planning

Determines the people, hardware, software, and other resources required.

5. Schedule planning

Determines when activities will start and finish.

6. Risk planning

Identifies possible problems and defines responses.

7. Quality planning

Defines quality standards, testing activities, reviews, and acceptance criteria.

Software Pricing

4.1 Definition

Software pricing is the process of determining how much a customer should pay for a software system or software development service.

Software pricing is not always the same as software development cost.

For example:

Development cost = ₹20 lakh

The company may charge:

Price = ₹30 lakh

The difference may cover profit, risk, support, marketing, and other business factors.

Factors Affecting Software Pricing

1. Development cost

Higher development costs generally lead to higher prices.

2. Software size

A larger system with more functionality generally costs more.

3. Complexity

Complex algorithms, integrations, security requirements, or performance requirements increase cost.

4. Development time

Longer development periods may increase labor and infrastructure costs.

5. Team size

More developers and specialists increase project expenditure.

6. Required quality

Software requiring high reliability, safety, security, or performance may require additional testing and development effort.

7. Customer requirements

Specialized requirements can increase development cost.

8. Market conditions

Competitor prices and customer demand influence the final price.

9. Business strategy

A company may deliberately price a product:

- Lower to enter a new market.
- Higher to target premium customers.
- According to the value delivered.

10. Project risks

Uncertain requirements and technical risks may be included in the price as a risk margin.

Software Pricing Strategies

A. Cost-based pricing

The price is calculated from development cost plus profit.

$$\text{Price} = \text{Cost} + \text{Profit}$$

Example:

Development cost = ₹10 lakh

Profit = ₹3 lakh

Price = ₹13 lakh

B. Market-based pricing

The price is influenced by competitor prices.

C. Value-based pricing

The price depends on the business value the software provides to the customer.

For example, software that saves a company ₹1 crore per year may be priced significantly higher than a simple application.

D. Fixed-price contract

A fixed amount is agreed upon before development.

Advantages:

- Customer knows the expected price.
- Easier budgeting.

Disadvantages:

- Risk is high when requirements are uncertain.
- Changes may result in disputes or additional charges.

E. Time-and-materials contract

The customer pays based on the actual effort and resources used.

Advantages:

- Flexible requirements.
- Suitable for Agile development.

Disadvantages:

- Final cost may be uncertain.
- Customer carries more cost risk.

6. Plan-driven Development



Plan-driven development defines a detailed plan before major implementation begins.

Suitable when requirements are stable, regulated or contractually fixed.

Strength: predictable documentation and milestones. Limitation: changes can be costly later.

It is commonly associated with traditional or **Waterfall-style development**.

Typical sequence:

Requirements → Design → Implementation → Testing → Deployment

Characteristics

- Extensive upfront planning
- Clearly defined requirements
- Formal documentation
- Sequential development
- Strong project management and control
- Changes are usually managed through formal change-control procedures

Advantages

- Clear project structure
- Easier to create a long-term schedule
- Easier to estimate total cost initially
- Strong documentation
- Suitable when requirements are stable
- Useful in regulated environments

Disadvantages

- Difficult to accommodate changing requirements
- Problems may be discovered late
- Customer feedback may arrive late
- Initial estimates can be inaccurate
- Working software may not be available until relatively late

When Plan-Driven Development Is Appropriate

It is useful when:

Requirements are well understood.

Requirements are unlikely to change.

The project is heavily regulated.

Extensive documentation is required.

The technology is well understood.

Contractual requirements demand detailed specifications.

7. Project Scheduling

Project Scheduling

Project scheduling determines **when project activities will occur and how long they will take.**

A schedule normally identifies:

- Tasks
- Task duration
- Dependencies
- Start dates
- End dates
- Responsible people
- Milestones
- Deliverables

Schedule — Food Delivery App

Week 1–2: Requirements & UI prototype

Week 3–5: Customer module

Week 4–6: Restaurant/order module

Week 5–7: Payment & notification integration

Week 7–8: System testing & bug fixing

Week 9: Deployment & training

Steps in Project Scheduling

Step 1: Identify Activities

Break the project into manageable activities.

Example:

- Requirements analysis
- UI design
- Database design
- Coding
- Unit testing
- Integration testing
- System testing
- Deployment

Step 2: Determine Dependencies

Some activities cannot start until others are completed.

Example:

Requirements → Design → Coding → Testing

Step 3: Estimate Duration

Estimate how long each activity will take.

Example:

Activity	Duration
Requirements	2 weeks
Design	3 weeks
Coding	6 weeks
Testing	3 weeks
Deployment	1 week

Step 4: Allocate Resources

Assign:

- Developers
- Testers
- Designers
- Project managers
- Infrastructure
- Equipment

Step 5: Identify Milestones

A milestone represents an important project event.

Examples:

- Requirements approved
- Design completed
- First prototype delivered
- Beta release
- Final release

Step 6: Create the Schedule

Scheduling tools may include:

- Gantt charts
- Network diagrams
- Critical Path Method
- PERT
- Agile release plans

Agile Planning

Agile planning is an iterative approach in which planning is performed continuously as the project evolves.

Unlike detailed long-term planning, Agile generally emphasizes **short-term, adaptive planning**.

Main characteristics

- Iterative development
- Incremental delivery
- Frequent customer feedback
- Continuous reprioritization
- Flexible requirements
- Short development cycles
- Planning at multiple levels

Agile Planning Levels

Product Vision

Describes the overall purpose of the product.

Example:

Build an online learning platform that enables students to access courses, assignments, and assessments.

Product Roadmap

Shows major product goals and expected releases.

Example:

- Release 1: User registration
- Release 2: Course management
- Release 3: Online assessments
- Release 4: Analytics

Release Planning

Determines which features are targeted for a particular release.

Iteration/Sprint Planning

Determines the work to be completed during the next iteration.

Daily Planning

Teams coordinate work during daily meetings.

Backlog Refinement

User stories are:

- Clarified
- Estimated
- Prioritized
- Split into smaller stories when necessary

User Stories

A **user story** describes functionality from the user's perspective.

Common format:

As a [type of user],
I want [functionality],
so that [benefit].

Example:

As a customer, I want to reset my password so that I can regain access to my account.

User stories are typically accompanied by **acceptance criteria** defining when the story is considered complete.

Product Backlog

The **product backlog** is a prioritized list of work required for the product.

It can contain:

- User stories
- Features
- Bugs
- Technical work
- Improvements
- Research tasks

Items near the top are generally more important and better understood than lower-priority items.

Agile Estimation

Agile teams commonly estimate work using **relative size** rather than attempting to predict exact hours for every task.

Common units include:

- Story points
- Ideal days
- T-shirt sizes

Story Points

Story points represent relative effort considering factors such as:

- Complexity
- Amount of work
- Uncertainty
- Technical difficulty

Example:

Story	Story Points
Login	2
Profile management	3
Payment integration	8
Recommendation engine	13

Estimation Techniques

Software estimation attempts to determine:

- Effort
- Development time
- Cost
- Resources

Estimation is difficult because software projects contain uncertainty.

Expert Judgment

Experienced developers/managers estimate the required effort based on their knowledge.

Advantages

- Fast
- Simple
- Useful when historical data is limited

Disadvantages

- Subjective
- Can be affected by bias
- Different experts may produce different estimates

Estimation by Analogy

The current project is compared with similar completed projects.

Example:

Previous project:

- 10,000 lines of code
- 12 person-months

New project appears approximately 25% larger.

Estimated effort might be around:

$$\mathbf{12 \times 1.25 = 15 \text{ person-months}}$$

Advantages

- Uses real historical experience
- Easy to understand

Disadvantages

- Requires sufficiently similar historical projects
- Differences between projects can make estimates inaccurate

Top-Down Estimation

The project is estimated as a whole first, and the total estimate is then divided among components.

Example:

Total estimated effort = 100 person-months

Then:

- Requirements = 10%
- Design = 15%
- Coding = 40%
- Testing = 25%
- Deployment = 10%

Advantages

- Quick
- Useful in early project stages

Disadvantages

- Individual tasks may be underestimated.
- Details may be overlooked.

Bottom-Up Estimation

The project is divided into smaller tasks, each task is estimated, and the estimates are added.

Example:

Login	2 days
Registration	3 days
Payment	5 days
Reporting	4 days
Testing	6 days

Total	20 days

Advantages

- Detailed
- Often more accurate when requirements are understood
- Makes assumptions visible

Disadvantages

- Time-consuming
- Difficult early in the project
- Small estimation errors can accumulate

Three-Point Estimation

Three estimates are produced:

- **Optimistic (O):** best-case estimate
- **Most likely (M):** normal expected estimate
- **Pessimistic (P):** worst reasonable estimate

A commonly used PERT-style expected estimate is:

$$E = \frac{O + 4M + P}{6}$$

Example

Suppose:

- $O = 4$ days
- $M = 6$ days
- $P = 10$ days

Three-point Estimation — Example

Suppose login module estimates are: O = 2 days, M = 4 days, P = 8 days.

Expected effort = $(2 + 4 \times 4 + 8) / 6 = 26/6 \approx 4.33$ days.

This method considers uncertainty rather than relying on one number.

Add estimates for all major work packages to obtain a project-level estimate.

Planning Poker

Planning Poker is an Agile estimation technique.

Process

- 1.Product owner explains a user story.
- 2.Team discusses the requirements.
- 3.Each team member privately selects an estimate.
- 4.Everyone reveals the estimate simultaneously.
- 5.High and low estimates are discussed.
- 6.The team estimates again.
- 7.The process continues until a reasonable consensus is reached.

Typical cards use values such as:

1, 2, 3, 5, 8, 13, 21

These values often follow a Fibonacci-like sequence because uncertainty increases as work becomes larger.

Benefits

Encourages team participation

Reduces influence from dominant individuals

Encourages discussion

Makes uncertainty visible

Function Point Estimation

Function Point Analysis (FPA) estimates software size based on functionality delivered to users rather than directly counting lines of code.

Common function types include:

- External Inputs (EI)
- External Outputs (EO)
- External Inquiries (EQ)
- Internal Logical Files (ILF)
- External Interface Files (EIF)

A simplified process is:

1. Identify functions.
2. Classify their complexity.
3. Assign weights.
4. Calculate unadjusted function points.
5. Apply adjustment factors if the chosen FPA method requires them.

Function points can then be related to historical productivity data to estimate effort.

Advantages

- Can be used before implementation.
- Independent of a specific programming language.
- Focuses on user-visible functionality.

Disadvantages

- Requires trained estimators.
- Classification can be subjective.
- More difficult for some modern systems.

LOC-Based Estimation

LOC = Lines of Code

Effort is estimated using expected source-code size and historical productivity.

For example:

If productivity is:

500 LOC/person-month

and estimated size is:

5,000 LOC

then:

Effort = $5000 / 500 = 10$ person-months

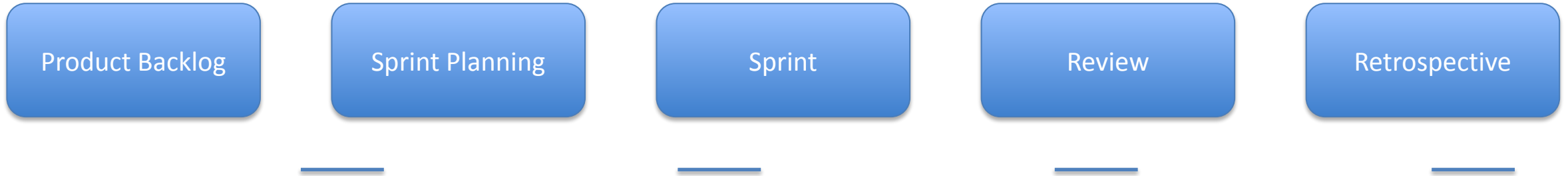
Problems with LOC

- Difficult to estimate LOC early.
- Different programming languages produce different amounts of code.
- More code does not necessarily mean more functionality.
- Encouraging more LOC can create undesirable incentives.

9. Estimation Techniques

- Expert judgment: experienced people estimate effort based on similar work.
- Analogous estimation: compare with a previous similar project.
- Top-down estimation: estimate the whole project, then distribute effort.
- Bottom-up estimation: estimate individual tasks/work packages and add them.
- Three-point estimation: optimistic (O), most likely (M), pessimistic (P). Expected estimate = $(O + 4M + P) / 6$.
- Function Point / Use Case Point methods estimate based on system functionality.

8. Agile Planning



Agile planning is iterative: plan a small amount of work, build it, get feedback and re-plan.

Prioritize work by business value and risk.

Common levels: product vision → release planning → sprint planning → daily planning.

Example: Sprint 1 delivers registration/login; Sprint 2 delivers restaurant search; Sprint 3 delivers cart and ordering.

End-to-End Real Example: Online Food Delivery



Need: order food online and track delivery.

Functional: login, search, cart, order, payment, tracking.

Non-functional: security, performance, availability, usability.

Risks: payment outage, inaccurate location, peak-load failure.

Planning: Agile sprints with prioritized backlog.

Estimation: task-based + three-point estimates; schedule reviewed every sprint.